



دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)

برنامه سازی پیشرفته و کارگاه

شی‌گرایی (چندریختی)

نگارش:

سام قربانی، آرمان حسینی، محمدحسین هاشمی، کیانا پهلوان، کیانا
رضوی، پیام علی‌مرادپور، مهدی جعفری

استاد درس:

دکتر احمدیان

دانشکده ریاضی و علوم کامپیوتر

دانشگاه صنعتی امیرکبیر

بهار ۱۴۰۵

فهرست مطالب

۲	۱	شی‌گرایی (چندریختی)
۲	۱.۱	مقدمه
۲	۲.۱	Polymorphism
۳	۱.۲.۱	Compile-time Polymorphism (Method Overloading)
۳	۲.۲.۱	Run-time Polymorphism (Method Overriding)
۴	۳.۱	interface
۵	۱.۳.۱	اینترفیس چیست و چطور از آن استفاده کنیم؟
۸	۲.۳.۱	کاربردهای interface
۹	۴.۱	مقایسه abstract classes و interfaces
۱۰	۱.۴.۱	Multiple inheritance by Interfaces
۱۳	۲.۴.۱	بعضی اینترفیس‌های مهم در جاوا
۱۳	۵.۱	Casting در جاوا
۱۷	۱.۵.۱	انواع casting
۱۷	۲.۵.۱	کست کردن در reference type ها
۲۴	۶.۱	کلاس Object
۲۴	۱.۶.۱	متدهای ناآشنا
۲۶	۲.۶.۱	تعریف کلاس Object
۲۷	۷.۱	چه چیزی یاد گرفتیم؟

فصل ۱ شی‌گرایی (چندریختی)

۱.۱ مقدمه

در این داکيومنت به مفاهيم پیشرفته‌تری از شی‌گرایی می‌پردازیم. چندریختی (Polymorphism) یکی از این مفاهيم است که این امکان را می‌دهد که متدهایی با نام‌های یکسان را به شکل‌های متفاوت پیاده‌سازی کنیم. اینترفیس نیز به کلاس‌های متفاوت، این امکان را می‌دهد که متدهای مشترکی را بدون به اشتراک گذاشتن کد و جزئیات پیاده‌سازی، داشته باشند. اینترفیس‌ها به ما کمک می‌کنند برنامه ساختارمندتر و منعطف‌تری داشته باشیم. حتماً کدهای این داک را اجرا کنید و هر سوالی که داشتید از تدریس‌یارها بپرسید، در مورد آن سرچ کنید و از DeepSeek راجع بهش اطلاعات بخواهید.

۲.۱ Polymorphism

در دنیای واقعی وقتی اطرافتان را نگاه می‌کنید، می‌بینید که خیلی از کارهای یکسان در موجودات مختلف به شکل‌های متفاوتی انجام می‌شود. مثلاً راه رفتن را در نظر بگیرید؛ هم اسب، هم انسان هر دو راه می‌روند، اما میدانیم که در انجام آن بسیار متفاوت‌اند یا مثلاً غذا خوردن، حرف زدن و خصوصیات دیگری که در موجودات مشترک‌اند اما در جزئیات و عملکرد متفاوت‌اند. حال در دنیای برنامه‌نویسی شی‌گرا، اگر بخواهید یک متد، کلاس یا شیء در شرایط مختلف رفتارهای متفاوتی داشته باشد، از قابلیت به اسم Polymorphism استفاده می‌کنید. این ویژگی باعث می‌شود کد شما منعطف‌تر، خواناتر و نگهداری آن راحت‌تر شود. جاوا دارای دو نوع پلی مورفیسم است:

• Compile-time Polymorphism (Method Overloading)

در این حالت چند متد با یک اسم ولی با ورودی‌های متفاوت در یک کلاس تعریف می‌کنید. این کار به شما اجازه می‌دهد که یک کار مشابه را با ورودی‌های مختلف انجام بدهید.

• Run-time Polymorphism (Method Overriding)

این یعنی یک متد در کلاس والد تعریف شده است و بعد در کلاس فرزند با همان اسم ولی پیاده‌سازی متفاوت بازنویسی می‌شود. این کار به شما اجازه می‌دهد رفتاریه متد را در کلاس‌های فرزند تغییر بدهید.

ایده اصلی این است که متدهایی با اسم یکسان می‌توانند رفتارهای متفاوتی داشته باشند و همین باعث می‌شود کد شما هم تمیزتر و هم راحت‌تر قابل استفاده مجدد باشد.

۱.۲.۱ Compile-time Polymorphism (Method Overloading)

گاهی وقت‌ها پیش می‌آید که چند متد با یک اسم در یک کلاس تعریف می‌کنیم، اما امضای (تعداد یا نوع پارامترهای) متدها با هم متفاوت اند. این کار باعث می‌شود وقتی متد را صدا می‌زنیم، کامپایلر بفهمد کدام متد باید اجرا شود. به این روش Method Overloading می‌گویند. ویژگی‌های این نوع polymorphism، به شکل زیر است:

- در یک کلاس انجام می‌شود.

- تفاوت در تعداد یا نوع پارامترها وجود دارد.

- در زمان کامپایل مشخص می‌شود.

مثلاً در کد زیر، چند متد add داریم که هر کدام، ورودی‌های متفاوتی می‌گیرند و خروجی متفاوتی می‌دهند:

```
class MathOperations {
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
}

public class Main {
    public static void main(String[] args) {
        MathOperations math = new MathOperations();
        System.out.println(math.add(5, 10)); // Output: 15
        System.out.println(math.add(5.5, 2.2)); // Output: 7.7
        System.out.println(math.add(1, 2, 3)); // Output: 6
    }
}
```

۲.۲.۱ Run-time Polymorphism (Method Overriding)

اینجا ایده این است که یک متد توی کلاس Parent داریم و به کلاس Child که می‌خواهد آن متد را به شکل خودش پیاده‌سازی کند. این کار را با Method Overriding انجام می‌دهیم.

ویژگی‌های این نوع، polymorphism به شکل زیر است:

- بین کلاس Parent و Child اتفاق می‌افتد.
 - امضای متد (شامل اسم آن، جنس ورودی‌ها و خروجی‌ها) تغییری نمی‌کند.
 - در زمان اجرای برنامه، بسته به تایپ متغیر تصمیم گرفته می‌شود کدام متد اجرا شود.
- مثلا در کد زیر، متد makeSound در Dog دوباره تعریف شده است. لازم است که وقتی متدی را override می‌کنید، حتما بالای آن از @Override استفاده کنید، وگرنه IntelliJ به شما هشدار می‌دهد.

```
class Animal {
    void makeSound() {
        System.out.println("An animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void makeSound() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal myAnimal = new Animal();
        myAnimal.makeSound(); // Output: An animal makes a sound

        Animal myDog = new Dog();
        myDog.makeSound(); // Output: Dog barks
    }
}
```

۳.۱ interface

ارث‌بری و وراثت ابزار بسیار کاربردی که به شما اجازه می‌دهد از polymorphism (چند ریختی) استفاده کنید ولی خب اگر توجه کرده باشید، محدودیتی هم جلوی شما قرار می‌دهد. جاوا از ارث‌بری یگانه (single inheritance) استفاده می‌کند، به این معنا که یک کلاس می‌تواند فقط یک superclass را extend کند. این محدودیت، اجازه نمی‌دهد که از ارث‌بری برای کلاس‌هایی استفاده کنیم که نیاز دارند شاخصه‌های متفاوت موجود در سوپرکلاس‌های مختلف را همزمان داشته باشند. امتحان کنید و ببینید که اگر بخواهید همزمان چند تا superclass را extend کنید، به کامپایل‌ارور بر می‌خورید.

همچنین، در بعضی موقعیت‌ها، ممکن است نخواهید که جزییات پیاده‌سازی و کد را به اشتراک بگذارید. اینجاست که با مفهوم جدیدی به اسم interface آشنا میشویم تا محدودیت‌های قبلی را کنار بگذاریم.

۱.۳.۱ اینترفیس چیست و چطور از آن استفاده کنیم؟

اینترفیس مثل کلاس‌های عادی است، صرفاً یکسری تفاوت‌ها و محدودیت‌ها دارد. Interface متشکل شده از یکسری تعریف اولیه از متدها است. متدهایی که بدنه ندارند و جزییات پیاده‌سازی آنها نوشته نشده است.

کلاس‌ها، سوپرکلاس‌های عادی یا abstract را extend می‌کردند و ویژگی‌های آنها را به ارث می‌بردند، ولی interface ها را implement (پیاده‌سازی) می‌کنند. Implement کردن یک اینترفیس به این معناست که کلاسی که می‌خواهد interface مورد نظر را پیاده‌سازی کند، قول می‌دهد که بدنه همه متدهای تعریف شده در اینترفیس را پیاده‌سازی کند (در غیر این صورت به کامپایل ارور می‌خورد). به عنوان مثال، می‌خواهیم یک اینترفیس به نام shape بسازیم. تصور کنید که می‌خواهیم کلاس‌هایی را بسازیم که هر کدام نماینده یک شکل متفاوت باشند (مثلاً مستطیل و دایره). ارث‌بری ایده خوبیست چون که هر دوی این اشکال رفتار مشابهی دارند (مثل مساحت و محیط) ولی نحوه محاسبه این مساحت و محیط (پیاده‌سازی متدها)، متفاوت و مرتبط با نوع شکل است. به این شکل، سلسه مراتب پولی‌مورفیسمی از کلاس‌های shape داریم بدون اینکه کدی و پیاده‌سازی‌ای بین آن‌ها به اشتراک گذاشته شده باشد.

برای نوشتن یک اینترفیس، از کلمه کلیدی interface استفاده می‌کنیم:

```
public interface Shape {
}
```

در داخل بدنه interface، متدها را مینویسیم ولی بجای اینکه از {} استفاده کنیم و داخل braces پیاده‌سازی متد را بنویسیم، به یک ; اکتفا می‌کنیم:

```
public interface Shape {
    public double getArea();
    public double getPerimeter();
}
```

خب ما الان اولین اینترفیسمون را ساختیم. توجه کنید که **هم اینترفیس هم class abstract** قابلیت این رو ندارند که از آنها آجکت ساخته شود. علت این موضوع، وجود abstract methods در هر دوی آنها است. پس اول باید یک کلاس، کلاس abstract را به ارث ببرد و یا اینترفیس را پیاده سازی کند تا بعد به صورت غیر مستقیم، بتواند از کلاس ثانویه آجکت ایجاد کند. حالا می‌خواهیم دو کلاس Rectangle و Circle را بسازیم که اینترفیس shape را implement کنند. هنگام تعریف این کلاس‌ها، باید از کلمه کلیدی implements استفاده کنیم. دقت کنید که این دو کلاس را باید در دو فایل جدا تعریف کنید:

```
public class Rectangle implements Shape {

}

public class Circle implements Shape {

}
```

وقتی که در تعریف این دو کلاس، اینترفیس shape را implement کردیم، تضمین کردیم که متدهای `getArea()` و `getPerimeter()` را پیاده‌سازی کنیم و تا زمانی که این کار را نکنیم، با کامپایل ارور روبه‌رو می‌شویم. کلاس‌ها را به صورت زیر گسترش می‌دهیم:

```
public class Rectangle implements Shape{
    private double width;
    private double height;
    public Rectangle(double width, double height){
        this.width = width;
        this.height = height;
    }

    @Override
    public double getArea() {
        return width * height;
    }

    @Override
    public double getPerimeter() {
        return 2.0 * (height + width);
    }
}

public class Circle implements Shape{
    private double radius;

    public Circle(double radius){
        this.radius = radius;
    }

    @Override
    public double getArea() {
```

```

        return Math.PI * radius * radius;
    }

    @Override
    public double getPerimeter() {
        return 2.0 * Math.PI * radius;
    }
}

```

الان کلاس‌های ما کامل شدند و میتوانیم ازشون استفاده کنیم و از روی آن‌ها آبجکت بسازیم. به مثال زیر دقت کنید:

```

public class ShapeMain {
    public static void main(String[] args){
        Shape circle = new Circle(5);
        Shape rectangle = new Rectangle(3, 6);
        System.out.println("rectangle's area = " + rectangle.getArea());
        System.out.println("rectangle's perimeter = " +
        ↪ rectangle.getPerimeter());
        System.out.println("circle's area = " + circle.getArea());
        System.out.println("circle's perimeter = " + circle.getPerimeter());
    }
}

```

خروجی کد بالا:

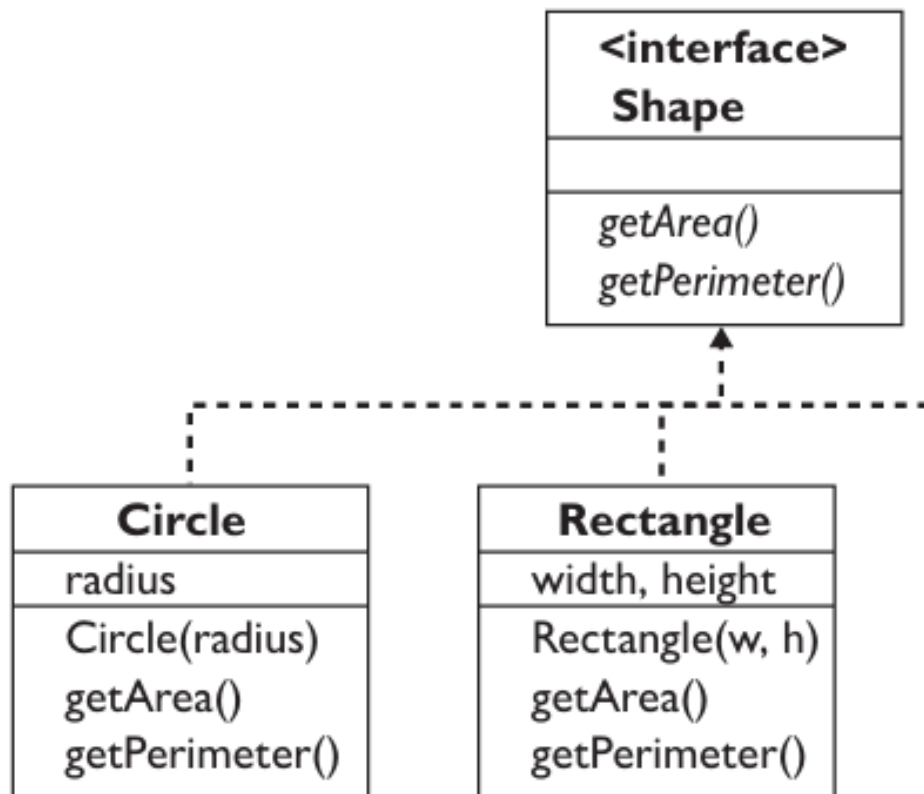
```

rectangle's area = 18.0
rectangle's perimeter = 18.0
circle's area = 78.53981633974483
circle's perimeter = 31.41592653589793

```

در این مثال، یک آبجکت از کلاس Rectangle و یک آبجکت از کلاس Circle ساختیم که هر دو اینترفیس Shape را implement می‌کردند و پیاده‌سازی مخصوص به خود برای متدهای این اینترفیس را داشتن. به سینتکس دقت کنید.

کلاس‌هایی که اینترفیس یکسانی را implement میکنند، سلسله مراتبی شبیه به کلاس‌های فرزندی که یک کلاس پدر را به ارث می‌ببرند، دارند. مثلاً برای مثال بالا، میتوانیم چنین ارتباطی را در نظر بگیریم:



حال که میدانیم interface چیست. به بررسی کاربرد آن و اینکه از آن در چه جاهایی استفاده می‌شود می‌رسیم.

۲.۳.۱ کاربردهای interface

۱. اینترفیس مثل یک قرارداد یا پروتکل عمل می‌کند. در واقع یک ساختار و پروتوتایپ ارائه می‌دهد که کلاس‌های مرتبط با آن باید آن‌ها را داشته باشند. با این حال، هر کدام می‌توانند پیاده‌سازی مخصوص خودشان را ارائه دهند. به همین دلیل دسترسی بسیار مناسبی برای polymorphism فراهم می‌کند، چون هر کلاسی که اینترفیس را پیاده‌سازی می‌کند می‌تواند پیاده‌سازی منحصر به فرد خودش را برای متدهای آن اینترفیس داشته باشد. همچنین این ویژگی امکان انعطاف‌پذیری بیشتری را در طراحی سیستم‌ها فراهم می‌کند.

۲. با تعریف یک interface، جزئیات پیاده‌سازی و کد ارائه نمی‌شود و صرفاً یک قالب در اختیار قرار می‌گیرد.

۳. مهم‌تر از همه، **چندگانه بودن وراثت (multiple inheritance)** در استفاده از اینترفیس‌ها است. اینترفیس‌ها این امکان را فراهم می‌کنند که یک کلاس بتواند همزمان از چندین اینترفیس پیروی کرده و آن‌ها را implement کند، اما در عین حال فقط می‌تواند یک کلاس را extend کند.

۴. همچنین استفاده از اینترفیس‌ها باعث کاهش وابستگی بین کلاس‌ها می‌شود، زیرا دیگر کلاس‌ها به جزئیات پیاده‌سازی یکدیگر وابسته نیستند. این موضوع می‌تواند منجر به کدهای تمیزتر و قابل نگهداری‌تر شود.

۵. اینترفیس‌ها معمولاً برای جداسازی مسئولیت‌ها استفاده می‌شوند. به جای اینکه یک کلاس تمام رفتارها و ویژگی‌ها را در خود داشته باشد، اینترفیس‌ها امکان تقسیم مسئولیت‌ها را فراهم می‌کنند و این موضوع باعث می‌شود کدها قابل تست‌تر و نگهداری آن‌ها آسان‌تر شود.

حالا یک سوالی که پیش می‌آید این است که خب چرا اصلاً Abstract Class وجود دارد وقتی که اینترفیس می‌تواند تقریباً همان کار را انجام بدهد؟ چرا فقط یکی از این دو را نداریم؟ برای جواب دادن به این سوال، باید تفاوت‌ها را بررسی کنیم. چون که این دو، برای اهداف متفاوتی طراحی شدند و نمیتوانند کاملاً جایگزین همدیگر بشوند.

۴.۱ مقایسه abstract classes و interfaces

کلاس abstract برای ساختارهای سلسله‌مراتبی (Hierarchy) که اشتراکات زیادی دارند، مناسب است. در مقابل، اینترفیس برای قراردادهای عمومی بین کلاس‌های نامرتب استفاده می‌شود.

Interface	Abstract Class
Interface can have only abstract methods .	Abstract class can have abstract and non-abstract methods .
Interface supports multiple inheritance .	Abstract class doesn't support multiple inheritance .
Interface has only static and final variables .	Abstract class can have final, non-final, static, and non-static variables .

همونطور که در جدول بالا می‌بینید، اینترفیس‌ها و کلاس‌های abstract تفاوت‌های ملموسی دارند.

به جز تفاوت در نحوه وراثت و ارث‌بری، اینترفیس‌ها برخلاف کلاس‌های abstract، فقط میتوانند فیلدهای static و final داشته باشند. یعنی اینترفیس‌ها، متغیر نمونه (instance variable) ندارند و فقط در صورتی میتوانند فیلد داشته باشند که مقدارش ثابت و static باشد. یک فیلد مثل سن در یک اینترفیس فقط میتواند به قالب زیر تعریف بشود:

```
public static final int age = 20;
```

این محدودیت به دلایل زیر به وجود آمده است:

• اینترفیس‌ها برای «تعریف رفتار» ساخته شدند، نه «نگهداری وضعیت»:

اینترفیس‌ها یک قرارداد (Contract) برای کلاس‌های پیاده‌سازشون ارائه می‌کنند و نباید شامل وضعیت (State) یا داده‌های متغیر باشند. در اینترفیس‌ها، تمرکز بر روی متدهاست، نه متغیرهای نمونه. اگر اینترفیس‌ها اجازه‌ی داشتن متغیرهای نمونه رو داشتند، در عمل شبیه به کلاس‌های abstract می‌شدند که خلاف فلسفه‌ی اینترفیس است.

• اینترفیس‌ها نباید وابستگی به مقدارهای متغیر داشته باشند:

اگر اینترفیس‌ها میتوانند متغیر نمونه داشته باشند، هر کلاس پیاده‌سازی آنها باید این مقدار رو مدیریت می‌کرد که باعث نقض استقلال اینترفیس می‌شد.

• جلوگیری از مشکل «وراثت چندگانه» و «داده‌ی متناقض»:

اگر اینترفیس‌ها متغیرهای نمونه داشتند، مشکل تناقض داده‌ها به وجود می‌آمد. مثالی رو در نظر بگیرید که در آن، یه کلاس دو اینترفیس را پیاده‌سازی میکند که هر کدام دارای یه فیلد با نام یکسان هستند ولی مقادیر متفاوتی دارند.

یک نکته قابل توجه دیگر، این است که اینترفیس بر خلاف class abstract کانستراکتور ندارد. **حال که میدانیم این دو متفاوت اند و اینکه علت تفاوت چیست، می‌خواهیم ببینیم که کجا و چرا باید از کدام ساختار استفاده کنیم.**

Interface	Abstract class
اگر نیاز دارید یک کلاس از چند منبع مختلف ویژگی‌هایی دریافت کند، اینترفیس‌ها گزینه بهتری هستند.	اگر فقط به یک لایه وراثت نیاز دارید و مشخص است که کلاس‌های فرزند شباهت‌های زیادی در پیاده‌سازی بین خودشان دارند، یک کلاس ابسترکت می‌تواند مناسب‌تر باشد.
اگر فقط می‌خواهید یک قرارداد تعیین کنید که همه‌ی کلاس‌ها موظف باشند متدهایی را پیاده‌سازی کنند و لزوماً این پیاده‌سازی‌ها شبیه هم نیستند، اینترفیس گزینه بهتری است.	اگر برخی متدها باید در همه‌ی کلاس‌های فرزند یکسان باشند و نمی‌خواهید کد تکراری بنویسید، کلاس ابسترکت بهتر است.
اگر فقط متدهای مورد نیاز را تعریف می‌کنید و هیچ داده و فیلدی در کلاس پایه نیاز ندارید (نیاز به متغیرهای نمونه ندارید) اینترفیس بهترین انتخاب است.	اگر کلاس باید مقادیر پیش‌فرض یا داده‌ای را در خود نگه دارد، کلاس ابسترکت بهتر است. مزیت کلاس‌های ابسترکت این است که میتوانیم هم متغیرهای مشترک و هم متدهای مشترک با سوپرکلاس را تعریف کنیم و در صورت نیاز به تغییر پیاده‌سازی در یک کلاس فرزند، override کنیم.

1.4.1 Multiple inheritance by Interfaces

حالا می‌خواهیم یک مثال را بررسی کنیم که یک کلاس، چند تا اینترفیس مختلف را implement میکند: تصور کنید یک شرکت فناوری قصد دارد یک ربات چندمنظوره بسازد که بتواند:

۱. پرواز کند (مثل یک پهلپاد)

۲. شنا کند (مثل یک زیردریایی)

۳. صحبت کند و ارتباط برقرار کند (مانند یک دستیار هوشمند)

هر یک از این قابلیت‌ها شامل چندین رفتار است. ما این قابلیت‌ها را به عنوان اینترفیس‌ها تعریف می‌کنیم تا ربات بتواند از اونا به صورت ارث‌بری چندگانه استفاده کند:

```
// interface for flying feature
interface Flyable {
    void fly();           // start flyting
    void adjustAltitude(int height); // adjusting height if flying
    void land();          // start landing
}
```

```
// interface for swim feature
interface Swimmable {
    void swim();           // start swimming
    void adjustDepth(int depth); // adjusting depth of swim
    void stopSwimming(); // stopping the swimming process
}

// interface for speaking feature
interface Speakable {
    void speak(String message); // speaking
    void setLanguage(String language); // changing the language
    void processCommand(String command); // processing voice commands
}

package Robot;
import java.lang.String;

// The class which implements 3 interfaces
class SmartRobot implements Flyable, Swimmable, Speakable {

    private int altitude;
    private int depth;
    private String language;
    public SmartRobot(int altitude, int depth, String language){
        this.depth = depth;
        this.altitude = altitude;
        this.language = language;
    }

    //implementing fly methods
    @Override
    public void fly() {
        System.out.println("Robot is flying ...");
    }

    @Override
    public void adjustAltitude(int height) {
        this.altitude = height;
        System.out.println("Height has been set to " + height + " meters.");
    }

    @Override
```

```
public void land() {
    this.altitude = 0;
    System.out.println("Robot has landed successfully");
}

// implementing swim methods
@Override
public void swim() {
    System.out.println("Robot is swimming");
}

@Override
public void adjustDepth(int depth) {
    this.depth = depth;
    System.out.println("Depth has been set to" + depth + " meters");
}

@Override
public void stopSwimming() {
    System.out.println("Robot stopped swimming successfully");
}

// implementing speak methods
@Override
public void speak(String message) {
    System.out.println("Robot says: " + message);
}

@Override
public void setLanguage(String language) {
    this.language = language;
    System.out.println("Language has been change into " + language);
}

@Override
public void processCommand(String command) {
    System.out.println("Processing the command: " + command);
}
}
```

تو این مثال من همه بخش هارو پشت هم نوشتم ولی در اصل بهتره هر اینترفیس و کلاس رو تو

یه فایل جداگانه تعریف کنید.

۲.۴.۱ بعضی اینترفیس‌های مهم در جاوا

خیلی از کلاس‌های موجود در لایبرری‌های جاوا، همزمان یک سوپرکلاس را extend کرده و یک یا بعضاً چند اینترفیس را implement میکنند. برای مثال می‌توان به کلاس `PrintStream` اشاره کرد (کلاسی که `system.out` یک نمونه (instance) از آن است). این کلاس به این صورت تعریف می‌شود:

```
public class PrintStream extends FilterOutputStream implements Appendable,
    ↪ closeable {

}
```

یک دسته اینترفیس مهم دیگر هم که جاوا در اختیارتون می‌گذاره، اینترفیس‌های `Map`، `List` و `set` هستند که توسط پکیج `java.util` ارائه می‌شوند و با implement کردن آنها می‌توانید ساختمان داده‌هایی را توصیف کنید که به کمک آنها مجموعه‌ای از آبجکت‌ها را ذخیره کنید.

۵.۱ Casting در جاوا

قبل از ادامهٔ ارث‌بری، مقداری در مورد `cast` کردن صحبت می‌کنیم. اصطلاح `cast` کردن، برای تغییر تایپ متغیرها استفاده می‌شود. مثلاً این کد را ببینید:

```
public static void main(String[] args) {
    int a = 10;
    long b = a;

    System.out.println("a: " + a);
    System.out.println("b: " + b);
}
```

در این کد، متغیر `a` با تایپ `int` به متغیری از جنس `long` به اسم `b` تبدیل شده است. این تبدیل بدون هیچ مشکلی انجام شده چون بازهٔ اعدادی که متغیرهای `long` پوشش می‌دهند بسیار بزرگتر از اعداد `int` است. به خاطر درستی این `cast` خروجی این کد هم به شکل زیر است:

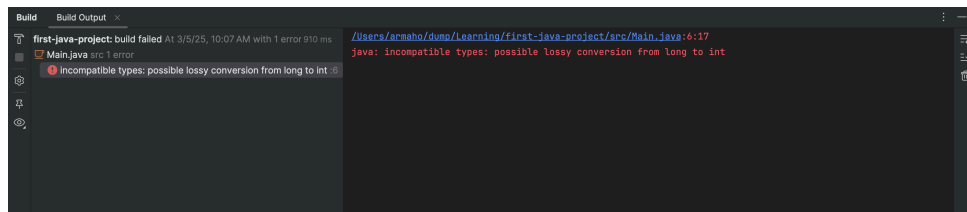
```
a: 10
b: 10
```

حالا اگر متغیر `long` را به یک متغیر `int`، کست کنیم چه اتفاقی می‌افتد؟ کد زیر را ببینید:

```
public static void main(String[] args) {
    long a = 10_000_000_000L;
    int b = a;

    System.out.println("a: " + a);
    System.out.println("b: " + b);
}
```

تلاش کنید این کد را اجرا کنید. به خطای زیر می‌خورید:



متن خطا می‌گوید:

```
java: incompatible types: possible lossy conversion from long to int
```

جاوا با این خطا به شما می‌گوید که تبدیل متغیرهای long به int، خطرناک است و می‌تواند منجر به خطاهای overflow شود. مثلاً در همین مثال، عدد 10000000000 به قدری بزرگ است که در یک متغیر int جا نمی‌شود و به همین خاطر تبدیل آن به یک int مشکلاتی برای شما به همراه دارد. حالا اگر یک int قبل از a قرار دهیم به جاوا می‌گوییم که از cast کردن مطمئنیم و می‌دانیم که هیچ مشکلی ایجاد نمی‌کند:

```
public static void main(String[] args) {
    long a = 10_000_000_000L;
    int b = (int)a;

    System.out.println("a: " + a);
    System.out.println("b: " + b);
}
```

خروجی این کد به شکل زیر است:

```
a: 10000000000
b: 1410065408
```

حالا به مقدار عجیب b نگاه کنید! مشکلی که می‌بینید، مشکل بسیار معروفی به اسم "overflow" یا "سرریزی" است. زمانی که شما، عدد خیلی بزرگی را داخل یک متغیر که فضای کافی ندارد ذخیره می‌کنید، به خطای overflow می‌خورید. خوب است بدانید مقدار چاپ شده 1410065408 دقیقاً برابر 31 بیت سمت راست 10000000000 می‌باشد. (یادآوری: int معمولاً 32 بیت در حافظه اشغال می‌کند؛ 1 بیت علامت و 31 بیت برای مقدار)

حالا اگر مقدار a کوچکتر بود به این خطا نمی‌خوردید:

```
public static void main(String[] args) {
    long a = 100;
    int b = (int)a;

    System.out.println("a: " + a);
    System.out.println("b: " + b);
}
```

خروجی این کد به شکل زیر است:

```
a: 100
b: 100
```

علت این که cast بالا به درستی انجام شد این است که مقدار 100 به راحتی در یک متغیر int جا می‌شود.

اتفاق مشابهی در cast کردن بین متغیرهای int و double هم رخ می‌دهد. Cast زیر به درستی انجام می‌شود:

```
public static void main(String[] args) {
    int a = 100;
    double b = a;

    System.out.println("a: " + a);
    System.out.println("b: " + b);
}
```

خروجی این کد به صورت زیر است:

```
a: 100
b: 100.0
```

ممیز خط دوم نشان می‌دهد که متغیر b از جنس double است.

ولی اگر تلاش کنید عددی اعشاری را به عددی صحیح تبدیل کنید، باز هم خطا می‌گیرید:

```
public static void main(String[] args) {
    double a = 3.14;
    int b = a;

    System.out.println("a: " + a);
    System.out.println("b: " + b);
}
```

دوباره با همان خطای قبلی روبه‌رو شدیم:

```
java: incompatible types: possible lossy conversion from double to int
```

این خطا به خاطر این است که در این `cast`، بخش اعشاری متغیر `a` از دست می‌رود. اگر شما این `cast` را با استفاده از `int` به صورت زیر انجام دهید:

```
int b = (int)a;
```

می‌بینید که فقط بخش صحیح متغیر `a`، در `b` ذخیره می‌شود:

```
a: 3.14
b: 3
```

حتی اگر دوباره متغیر `b` را به `double` تبدیل کنید، می‌بینید که بخش اعشاری برنمی‌گردد:

```
public static void main(String[] args) {
    double a = 3.14;
    int b = (int)a;
    double c = b;

    System.out.println("a: " + a);
    System.out.println("b: " + b);
    System.out.println("c: " + c);
}
```

خروجی این کد به شکل زیر است:

```
a: 3.14
b: 3
c: 3.0
```

این، cast، خطرناک محسوب می‌شود چون در اون، بخشی از داده‌ها از دست رفته‌اند.

۱.۵.۱ انواع casting

دیدید که cast کردن، می‌تواند خطرناک باشد. جاوا می‌داند که چه cast هایی می‌توانند مشکل‌زا باشند. اگر مثل مثال اول، یک متغیر int را به long تبدیل کنیم، جاوا با اطمینان از موفق و بی‌خطر بودن این cast، آن را بدون هیچ خطا و اختطاری اجرا می‌کند. اما در مثال دوم، که cast کردن می‌توانست مشکل‌زا باشد، جاوا به ما خطا داد و مجبور شدیم با استفاده از (int) به او بگوییم که ما نسبت به این مشکلات آگاهیم و همچنان می‌خواهیم این cast را انجام دهیم. به طور کلی cast هایی که ممکن است برای برنامه شما با از دست دادن اطلاعات، overflow یا هر خطای دیگری مواجه شوند، cast های خطرناکی محسوب می‌شوند.

بر همین اساس، cast ها را به دو دسته تقسیم می‌کنیم:

کست‌های implicit

واژه "implicit" یعنی چیزی که بدون اشاره مستقیم فهمیده می‌شود. مثل وقتی که شما لازم نیست با گذاشتن (int) مستقیماً به جاوا بگید که از خطرات cast آگاهید. cast هایی که مشکلی برای برنامه شما ایجاد نمی‌کنند، به cast های implicit معروف هستند. cast کردن متغیر int به long از همین نوع است.

کست‌های explicit

واژه "explicit" به معنای چیزی است که باید مستقیماً و به شکل دقیق بیان شود. کست‌های explicit، کست‌هایی هستند که ممکن است مشکل‌زا باشند و لازم است که در کدتان حتماً اعلام کنید که از انجام آن cast آگاهید و در جریان مشکلات احتمالی هستید. تبدیل کردن متغیر long به int از این جنس است.

۲.۵.۱ کست کردن در reference type ها

حالا به ارث‌بری برگردیم. اگر به خاطر داشته باشید، به شما گفته بودیم class ها، ابزاری هستند که به وسیله آن‌ها می‌توانید برای خودتان type های جدید تعریف کنید. ولی خب، آیا این type ها هم قابل cast کردن هستند؟ جواب طبیعتاً بله‌ست.

بیایید با یک مثال cast کردن در این type ها را ببینیم. کد زیر را در تابع main بنویسید:

```
public static void main(String[] args) {
    Car myCar = new Car("Iran-Khodro", 180, 4);
    myCar.showCarInfo();
}
```

آن را اجرا کنید تا مطمئن باشید همه چیز درست کار می‌کند:

```
Brand: Iran-Khodro, Speed: 180 km/h
Doors: 4
```

حالا، بیاید یک cast عجیب بین Car و Vehicle انجام بدیم:

```
public static void main(String[] args) {
    Car myCar = new Car("Iran-Khodro", 180, 4);
    myCar.showCarInfo();

    System.out.println();

    Vehicle myVehicle = myCar; // Casting from Car to Vehicle
    myVehicle.showInfo();
}
```

خروجی این کد، به شکل زیر است:

```
Brand: Iran-Khodro, Speed: 180 km/h
Doors: 4

Brand: Iran-Khodro, Speed: 180 km/h
```

اگر دقت کنید، وقتی که Car از Vehicle ارث‌بری می‌کند، تمام فیلدها و متدهای Vehicle متعلق به Car نیز هستند. یعنی هر آبجکتی از جنس Car، تمام فیلدهای یک Vehicle را در خودش دارد، پس ما همه چیز برای cast کردن Car به Vehicle را داریم و می‌توانیم آن را انجام دهیم. همان‌طور که می‌بینید، برند myVehicle ایران خودرو و سرعتش 180 km/h است. دقیقا مثل myCar. حتی می‌توانیم myVehicle را با یک دوچرخه هم جایگزین کنیم:

```
public static void main(String[] args) {
    Car myCar = new Car("Iran-Khodro", 180, 4);
    myCar.showCarInfo();

    System.out.println();

    Vehicle myVehicle = myCar; // Casting from Car to Vehicle
```

```

myVehicle.showInfo();

System.out.println();

Bike myBike = new Bike("Scott", 10, false);
myBike.showBikeInfo();

System.out.println();

myVehicle = myBike; // Casting, again!
myVehicle.showInfo();
}

```

می‌بینید که myVehicle از یک ماشین به یک دوچرخه تبدیل می‌شود. بیاید یک لحظه به اتفاق خیلی خوبی که الان دیدیم بیشتر فکر کنیم. دیدیم که متغیرهایی با جنس Vehicle، می‌توانند با آبجکت‌های Car، Bike و Motorcycle پر شوند. اتفاقی که بدون ارث‌بری ممکن نبود. فرض کنید بدون ارث‌بری، می‌خواستید متدی بنویسید که با توجه به سرعت یک وسیله نقلیه، بگوید یک مسافت مشخص در چند ثانیه طی می‌شود. اگر ارث‌بری نبود، مجبور بودید به شکل زیر، برای هریک از وسایل نقلیه متدی جدا تعریف کنید:

```

public static double calculateTimeForCar(double distance, Car car) {
    return distance / car.speed;
}

public static double calculateTimeForBike(double distance, Bike bike) {
    return distance / bike.speed;
}

public static double calculateTimeForMotorCycle(double distance, Motorcycle
↪ motorCycle) {
    return distance / motorCycle.speed;
}

```

می‌بینید که مقدار زیادی کد تکراری داریم و مجبور شدیم برای هر وسیله نقلیه، یک متد جدا بنویسیم. شرایط زمانی بدتر می‌شود که بخواهیم کلاس تریلی را هم اضافه کنیم، اما یادمان برود که متد محاسبه زمان بر حسب مسافت را برای آن بنویسیم. ارث‌بری کار ما را خیلی راحت می‌کند چون زمانی که همه کلاس‌ها از vehicle ارث‌بری کنند، می‌توانیم فقط یک متد برای محاسبه زمان داشته باشیم:

```
public static double calculateTime(double distance, Vehicle vehicle) {
    return distance / vehicle.speed;
}
```

و این متد، توسط تمام آبجکت‌هایی که از جنس Vehicle هستند، قابل استفاده است. این موضوع شامل آبجکت‌های Car، Bike، Motorcycle و حتی تریلی‌های آینده هم می‌شود!

```
public class Main {
    public static void main(String[] args) {
        Vehicle myVehicle = new Vehicle("Benz", 200);
        Car myCar = new Car("Iran-Khodro", 180, 4);
        Bike myBike = new Bike("Scott", 10, false);
        Motorcycle myMotorCycle = new Motorcycle("Yamaha", 100, false);

        System.out.println("time taken by myVehicle to move 10 km: " +
        ↪ calculateTime(10, myVehicle));
        System.out.println("time taken by myCar to move 10 km: " +
        ↪ calculateTime(10, myCar));
        System.out.println("time taken by myBike to move 10 km: " +
        ↪ calculateTime(10, myBike));
        System.out.println("time taken by myMotorCycle to move 10 km: " +
        ↪ calculateTime(10, myMotorCycle));
    }

    public static double calculateTime(double distance, Vehicle vehicle) {
        return distance / vehicle.speed;
    }
}
```

این کد، خروجی زیر را می‌دهد:

```
time taken by myVehicle to move 10 km: 0.05
time taken by myCar to move 10 km: 0.055555555555555555
time taken by myBike to move 10 km: 1.0
time taken by myMotorCycle to move 10 km: 0.1
```

explicit cast در reference type ها

اگر دقت کنید، در cast های بالا، لازم نبود که داخل پرانتز، explicitly مشخص کنیم تا جاوا بداند ما از مشکلات cast آگاهیم، چون cast هایی که انجام دادیم اصلاً مشکلی به وجود نمی‌آوردند! ولی همیشه این‌طور نیست. مثلاً اگر بخواهیم که یک Vehicle را به یک Car تبدیل کنیم چه می‌شود؟
کد زیر را ببینید:

```
public static void main(String[] args) {
    Car myCar = new Car("Iran-Khodro", 180, 4);
    myCar.showCarInfo();

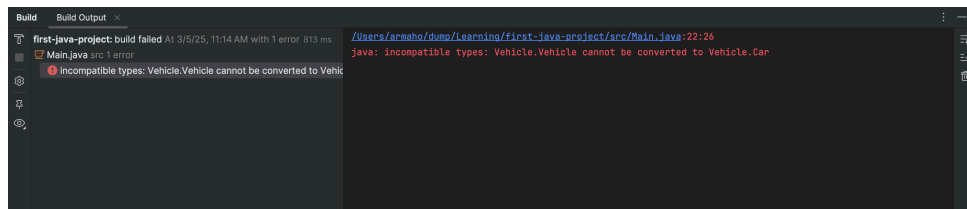
    System.out.println();

    Vehicle myVehicle = myCar;
    myVehicle.showInfo();

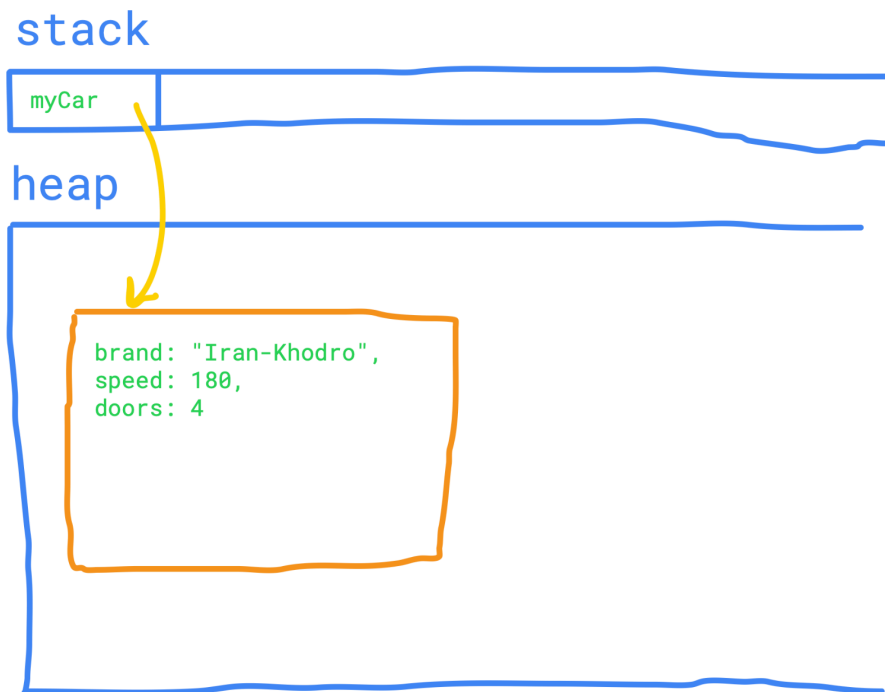
    System.out.println();

    Car theSameCar = myVehicle;
    theSameCar.showCarInfo();
}
```

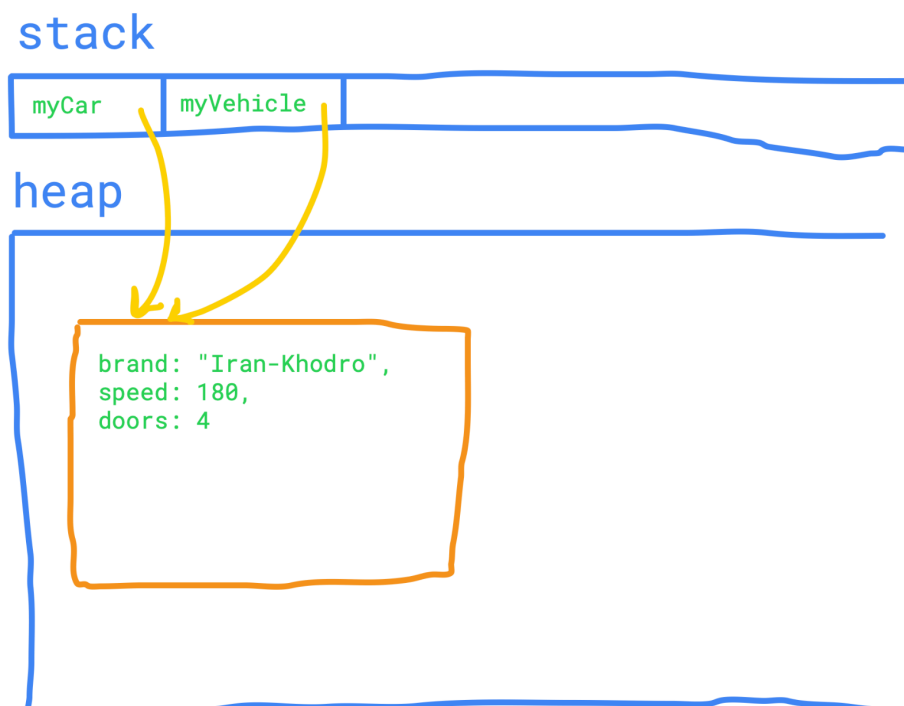
تلاش کنید آن را اجرا کنید. به همان خطای همیشگی می‌خورید:



چرا شما باید بتوانید Vehicle رو به Car تبدیل کنید؟! ممکن است Car فیلدهای اضافه نسبت به Vehicle داشته باشد. اما راه‌هایی برای انجام این cast وجود دارد. برای این که بفهمیم چرا و چگونه باید نگاهی به مموری بکنیم. اگر از بخش «reference type ها»ی داک قبلی به خاطر داشته باشید، بعد از درست شدن متغیر myCar و مقداردهی‌اش، مموری چنین شکلی دارد:



بعد از این که `myVehicle` هم ساخته شود، مموری به شکل زیر در می‌آید:



می‌توانید این موضوع را با تکه کد زیر تایید کنید. این جا می‌بینید که تغییر سرعت `myVehicle`، باعث تغییر سرعت `myCar` هم می‌شود:

```
myVehicle.accelerate(100);

System.out.println("myVehicle speed: " + myVehicle.speed);
System.out.println("myCar speed: " + myCar.speed);
```

خروجی این کد به شکل زیر است:

```
myVehicle speed: 280
myCar speed: 280
```

می‌بینید که سرعت هر دو دستگاه از 180 به 280 رسیده. حالا سراغ cast کردن می‌رویم:

```
Car theSameCar = myVehicle;
```

با این که جاوا اخطار مشکل‌زا بودن این cast را می‌دهد، ولی اگر باز هم به مموری نگاه کنیم، می‌بینیم که myVehicle هم به آبجکتی از جنس Car اشاره می‌کند! آبجکت car ای که myVehicle به آن اشاره می‌کند، همچنان فیلدی به اسم doors در مموری دارد و به همین خاطر، اگر به جاوا explicitly اطمینان دهیم که می‌دانیم چه کار می‌کنیم، این cast انجام می‌شود:

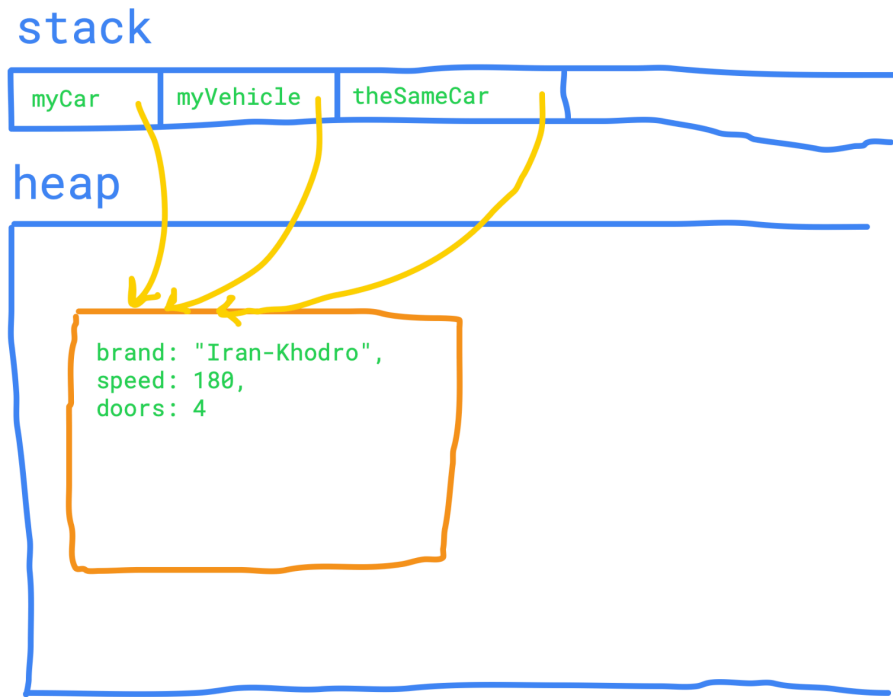
```
Car theSameCar = (Car)myVehicle;

theSameCar.showCarInfo();
```

اگر این کد را اجرا کنید، به خطایی نمی‌خورید و می‌بینید که theSameCar هم مثل myCar چهار در دارد:

```
Brand: Iran-Khodro, Speed: 280 km/h
Doors: 4
```

در نهایت مموری به شکل زیر خواهد بود:



همان‌طور که می‌بینید، هر سه متغیری که در این برنامه تعریف کردید، دقیقاً به یک‌جا اشاره می‌کنند.

۶.۱ کلاس Object

۱.۶.۱ متدهای ناآشنا

کد زیر را در main بنویسید:

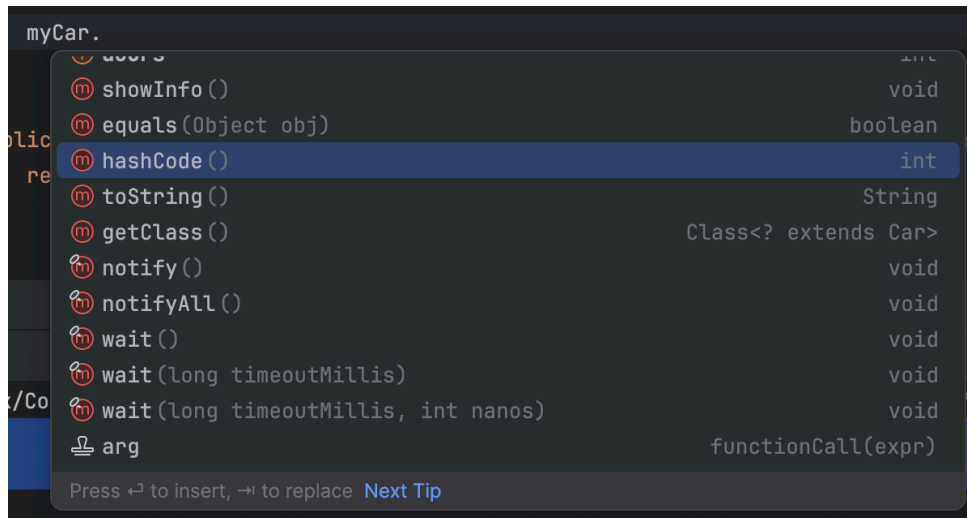
```
public static void main(String[] args) {
    Car myCar = new Car("Iran-Khodro", 180, 4);
    myCar.showCarInfo();

    System.out.println("Hash code: " + myCar.hashCode());
}
```

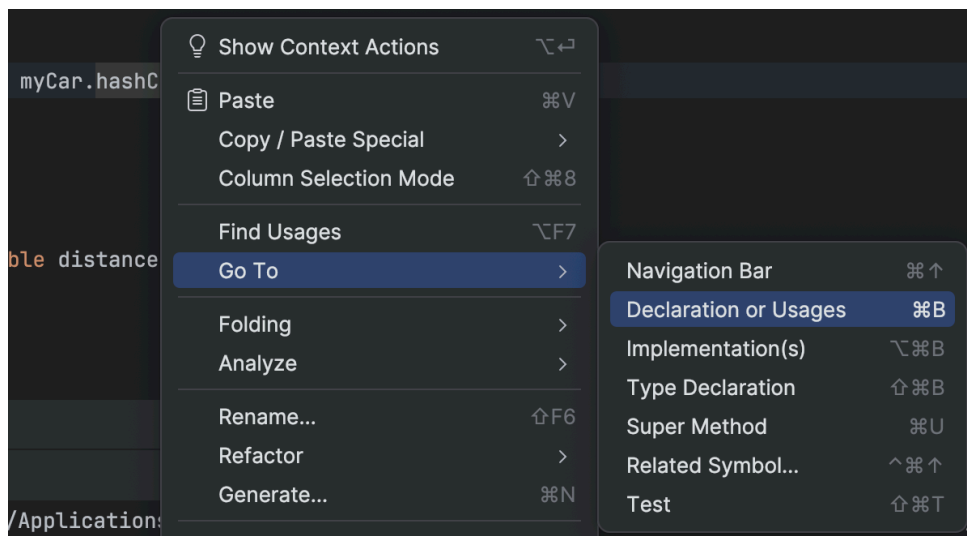
متد `hashCode()` در خط آخر را کجا تعریف کردیم؟! چطور بدون هیچ اخطار قبلی‌ای، از آن استفاده کردیم؟ خروجی این کد به شکل زیر است:

```
Brand: Iran-Khodro, Speed: 180 km/h
Doors: 4
Hash code: 500977346
```

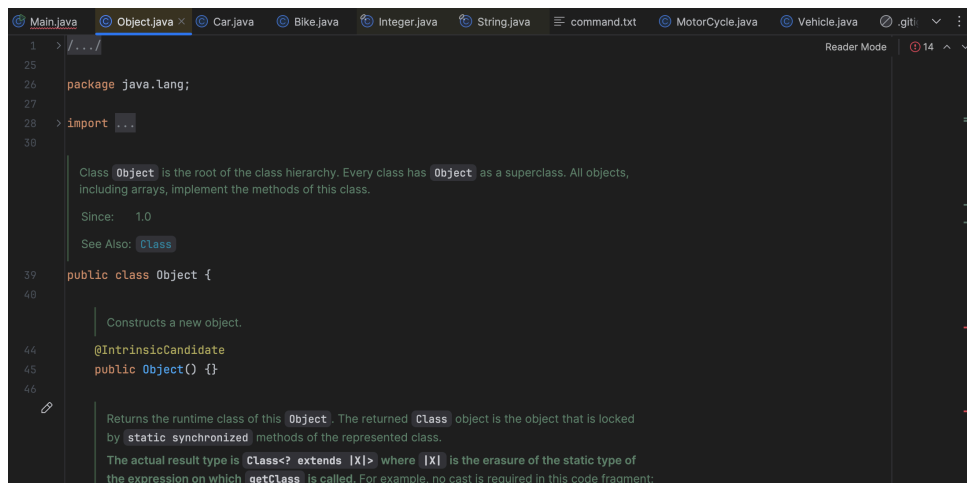
همه چیز عجیب‌تر هم می‌شود! myCar، چندین متد دیگر هم دارد که ما هیچ‌جا آن‌ها را تعریف نکردیم! حتی IntelliJ هم به ما پیشنهاد می‌دهد که از آن‌ها استفاده کنیم:



روی همین متد hashCode() کلیک راست کنید، از منوی Go To، گزینه Declaration or Usages را انتخاب کنید تا پیاده‌سازی این متد را ببینید:



اگر مقداری این صفحه را اسکرول کنید، می‌بینید که این متد، جزئی از کلاسی به اسم Object است:



۲.۶.۱ تعریف کلاس Object

حالا کلاس Object چی هست؟ Object کلاسی‌ست که همه کلاس‌های دیگر از آن به صورت مستقیم یا غیرمستقیم ارث‌بری می‌کنند. درکد شما، Vehicle از Object ارث‌بری می‌کند و Car هم از Vehicle، پس به شکلی غیرمستقیم از Object ارث‌بری می‌کند و به همین خاطر، به متدهای آن مثل hashCode()، toString() یا equals(Object obj) دسترسی دارد (دقت کنید که کلاس Car همچنان فقط یک پدر دارد و کلاس Object در واقع پدر بزرگ این کلاس است). شما تا الآن بدون این که خبر داشته باشید از این متدها استفاده می‌کردید. مثلاً وقتی کد زیر را بزنید:

```
System.out.println(myCar);
```

خروجی زیر را می‌بینید:

```
Vehicle.Car@1ddc4ec2
```

این output عجیب و غریب، در واقع خروجی متد myCar.toString() است. اگر کد زیر را اجرا کنید:

```
System.out.println(myCar.toString());
```

می‌بینید که خروجی هیچ تفاوتی ندارد:

```
Vehicle.Car@1ddc4ec2
```

حتی می‌توانید این موضوع را در پیاده‌سازی متد println(Object obj) هم ببینید. این متد به شکل زیر پیاده‌سازی شده است:

```
public void println(Object x) {
    String s = String.valueOf(x);
    if (getClass() == PrintStream.class) {
        // need to apply String.valueOf again since first invocation
        // might return null
        writeln(String.valueOf(s));
    } else {
        synchronized (this) {
            print(s);
            newline();
        }
    }
}
```

به جزئیات پیاده‌سازی این کد کاری نداریم فقط باید بدانیم که متد `writeln` در این کد output را برای شما چاپ می‌کند. می‌بینید که در این کد، `String.valueOf(x)` صدا زده شده. این متد هم پیاده‌سازی ساده‌ی زیر را دارد:

```
public static String valueOf(Object obj) {
    return (obj == null) ? "null" : obj.toString();
}
```

این متد، با ورودی گرفتن یک `Object`، اگر `null` باشد، رشته `null` و در غیر این صورت `obj.toString()` را برمی‌گرداند.

این‌جا هم می‌توانید کمکی که `inheritance` به ما می‌کند را ببینید. از آن‌جایی که متد `println` یک `Object` ورودی می‌گیرد، شما می‌توانید هر `reference type` دیگری مثل `Vehicle`، `Car` و هر کلاس دیگری را به آن ورودی بدهید. فرض کنید مجبور بودیم به خاطر عدم وجود `inheritance` برای هر کلاس یک بار `println` را پیاده‌سازی کنیم!

۷.۱ چه چیزی یاد گرفتیم؟

مهم‌ترین نکاتی که یاد گرفتیم:

- این که `interface` چیست و برای چه به آن نیاز داریم
- مقایسه `interface` ها و `abstract class` ها و اینکه چرا نمی‌توان به صورت مستقیم از آنها شیء ساخت
- خاصیت ارث‌بری چندگانه از اینترفیس‌ها
- آشنایی با انواع `polymorphism` ها و کاربرد آنها

- آشنایی با انواع و نحوهٔ **casting** در **primitive type** ها و **refrence type** ها و خطرات احتمالی آن
- اتفاقات درون حافظه هنگام **casting** در **refrences type** ها
- کلاس **object** چیست
- متدهای ناآشنایی که توسط ما تعریف نشده اند اما قابل استفاده اند، چه ارتباطی با کلاس **Object** دارند